

P1278R0

offsetof For the Modern Era

Published Proposal, 2018-10-06

This version:

<https://wg21.link/p1278r0>

Author:

[Isabella Muerte](#)

Audience:

LEWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Current Render:

[P1278R0](#)

Current Source:

slurps-mad-rips/papers/proposals/modern-offsetof.bs

Abstract

``offsetof()`` needs a modern take for C++.

Table of Contents

1 Revision History

1.1 Revision 0

2 Motivation

3 Design Considerations

4 Wording

4.1 Feature Testing

§ 1. Revision History

§ 1.1. Revision 0

Initial Release 🎉

§ 2. Motivation

`offsetof()` is a relic of its time. It's a macro, works only on standard layout types, and by its very definition requires a compiler to either implement it via an intrinsic or to use undefined behavior.

While it would be nice to assume that `offsetof` and similar tools are no longer needed, this is unfortunately not the case. When working with some C APIs, or older C++ APIs, knowing the offset of a type is necessary for a library to remain "ABI" safe. A good example of this is the Python scripting language. Python's C API allows users to "bind" C and C++ types to the Python API. Of important note is that data members can be bound to a Python class directly via the `PyMemberDef` struct. This struct requires type information about the "reflected" member in the form of a constant (such as `T_OBJECT` or `T_LONG`), and the offset to the given member. As using `offsetof()` can in some cases cause undefined behavior, it means that fully valid code is *technically speaking* not safe to work in a portable manner, thus upending Python's C API which is designed for portable use across platforms.

An example of this problem can be found below (actual final binding code has been removed for clarity):

```

#include <Python.h>
#include "structmember.h"

struct CustomObject {
    PyObject_HEAD
    PyObject *first;
    PyObject *last;
    int number;
};

static PyMemberDef Custom_members[] = {
    {"first", T_OBJECT_EX, offsetof(CustomObject, first), 0, "first name"},
    {"last", T_OBJECT_EX, offsetof(CustomObject, last), 0, "last name"},
    {"number", T_INT, offsetof(CustomObject, number), 0, "custom number"},
    { } /* Sentinel */
};

```

As we can see above, offsets for `first`, `last`, and `number` are all obtained via the `offsetof()` macro. However, this also places us into the predicament that we are unable to write a generic function to handle all of this work for us. Specifically, under standard C++, we cannot implement something equivalent to the following:

```

template <class T>
PyMemberDef readonly (PyObject* (T::*member), char const* name, char const* docs) {
    return { name, T_OBJECT_EX, offsetof(T, member), READONLY, docs };
}

template <class T>
PyMemberDef member (PyObject* (T::*member), char const* name, char const* docs) {
    return { name, T_OBJECT_EX, offsetof(T, member), 0, docs };
}

```

This paper attempts to fix this by creating a new function named `std::offset` that can take pointer to member data as a parameter directly, without needing to know the name of the actual member. `std::offset` is intended to be similar to `offsetof()` in the same way that `std::bit_cast` is similar to `std::memcpy`.

§ 3. Design Considerations

The design for `std::offset` is simple. Given any pointer to member data for a Standard Layout class, vendors will return the offset to said member according to the current target environment. In theory this sounds wild and unwieldy, however in practice, all current vendors simply place the offset to said member inside its pointer to member as either a `ptrdiff_t` or an `int`.

There is, however, one interesting caveat. `std::offset` can be implemented entirely via `std::bit_cast`. Doing so means that `std::offset` cannot be `constexpr`. This paper is not tackling changing the constraints found on `std::bit_cast`, but a discussion is needed on whether `std::offset` should be `constexpr`, and whether `std::bit_cast` should permit `constexpr` casting of pointer to member data from Standard Layout classes to a `ptrdiff_t`.

To match `offsetof`, it is the author's recommendation that `std::offset` be placed into the `<stddef>` header.

§ 4. Wording

The following is working for the library section.

```
namespace std {  
    template <class T>  
    ptrdiff_t offset (T const& pmd) noexcept;  
}
```

1. Constraints This function shall not participate in overload resolution unless:

- `std::is_member_object_pointer_v<T>` is true
- `std::is_standard_layout_v<class-of-T>` is true

Note: There is not `std::class_of_t<T>` type trait available in standard C++. The above is shown for exposition purposes only.

2. Returns

An object of type `std::ptrdiff_t`, the value of which is the offset in bytes from the beginning of an object of the *class-of* `T` to the member specified by `T` including padding if any.

3. Remarks

While `offsetof` is conditionally supported for non-standard layout types, `_offset` is intended to be more tightly constrained and *only* for standard layout types.

§ 4.1. Feature Testing

The `__cpp_lib_offset` feature test macro should be added.